

The Contextual Gatekeeper

Production Product Requirements Document (PRD) for Multi-Agent Triager

Author:	Tim (AI Architect)
Orchestration Framework:	LangGraph & LangSmith Tracing
Frontend Paradigm:	Streamlit UI (IBM Carbon Light)
Target Dev Environment:	Claude Code CLI + VSCode + uv
Version:	1.0.0

1. Product Vision & Operational Context

The Contextual Gatekeeper is a localized background workflow engine built to safely turn enterprise incoming communication channels (Gmail, Outlook) into a managed multi-agent event loop. By using modern design patterns, it ensures total modular insulation, deterministic human validation points, and robust diagnostic visibility.

2. Technical Stack Constraints

- **Package Management:** Fully standardized using uv commands (e.g., `uv pip compile` or `uv run`).
- **State Management:** Built entirely with LangGraph functional mappings, utilizing memory checkpoint states.
- **User Interface:** Streamlit serving an IBM Carbon-inspired minimalist canvas layout.

3. Component Architecture & Abstract Decoupling

To eliminate direct dependency bindings to precise vendor systems, communications and structural indexing must utilize abstract definitions.

3.1 Channel Decoupling Pattern

All core computational steps communicate strictly via an abstract provider class. This ensures switching from Google's API to Microsoft's graph involves no changes to node logic.

Security Mandate: Isolation of Mutations

No large language model output can execute direct API post routines without halting at a hard programmatic breakpoint boundary first.

3.2 Phase-Based Storage Strategy

- **Phase 1:** Storage utilizes local file directories or a compact local SQLite instance handled cleanly behind a single data repository class.
- **Phase 2:** Upgrading structural reference models to interface directly with PostgreSQL engines or live Google Sheets environments.

4. Modular File Topology

Claude Code must construct and test the codebase within this isolated folder structure:

```

contextual_gatekeeper/
├── .env                                # System secrets and LangSmith tokens
├── pyproject.toml                      # Managed by uv
├── config/
│   └── settings.py                    # Environment configurations
├── src/
│   ├── providers/                     # Abstract/Concrete Comms
│   │   ├── base.py                   # ICommunicationProvider
│   │   ├── google_prov.py            # Gmail Engine
│   │   └── ms_prov.py                # Outlook Stub
│   ├── database/                     # System state caching
│   │   └── local_store.py            # Local data adapter
│   ├── agents/                       # Computational Nodes
│   │   ├── state.py                  # LangGraph Core TypedDict State
│   │   ├── triage.py                 # Intake Node
│   │   ├── researcher.py             # Context Fetcher Node
│   │   └── drafter.py                # Draft Composer Node
│   └── graph.py                       # Graph DAG Compiler & Checkpointer
└── app/
    └── main.py                        # Streamlit Carbon Light UI

```

5. Interface Definitions & Graph States

5.1 Provider Model Interface (src/providers/base.py)

```

from abc import ABC, abstractmethod
from pydantic import BaseModel
from typing import List

class EmailMessage(BaseModel):
    id: str
    sender: str
    subject: str
    body: str
    received_at: str

class ICommunicationProvider(ABC):
    @abstractmethod
    def fetch_unread_emails(self, max_results: int = 10) -> List[EmailMessage]:
        pass

    @abstractmethod
    def send_reply(self, original_email_id: str, reply_body: str) -> bool:
        pass

```

5.2 Stateful Graph Core (src/agents/state.py)

```
from typing import TypedDict, Optional
from src.providers.base import EmailMessage

class GatekeeperState(TypedDict):
    current_email: Optional[EmailMessage]
    extracted_metadata: dict
    project_context: Optional[str]
    generated_draft: Optional[str]
    is_approved: bool
    user_modifications: Optional[str]
    rejection_reason: Optional[str]
```

6. Security Implementation Blueprint

Vector Protection	Mitigation Strategy	Implementation Pattern
Indirect Prompt Injection	Data Content Masking	Raw inbound message payloads are injected as Pydantic dictionary properties rather than string contexts for LLM completion sequences.
Runaway Execution Exception	Deterministic Halts	Compilation utilizes explicit interrupt_before=["send_email_node"] definitions.

7. Incremental Implementation Path

Development is broken down sequentially for optimal performance inside Claude Code CLI:

- **Step 1:** Build environment matrices, uv definitions, and abstract class files.
- **Step 2:** Code complete core graph.py using placeholder validation stubs to verify programmatic halts.
- **Step 3:** Wire GMail authorization layers into the communication block.
- **Step 4:** Render the custom Streamlit layout using explicit component flags.
- **Step 5:** Export state logs into LangSmith platforms for runtime evaluation.